CEFET-MG | DEPARTAMENTO DE COMPUTAÇÃO

Computação Gráfica

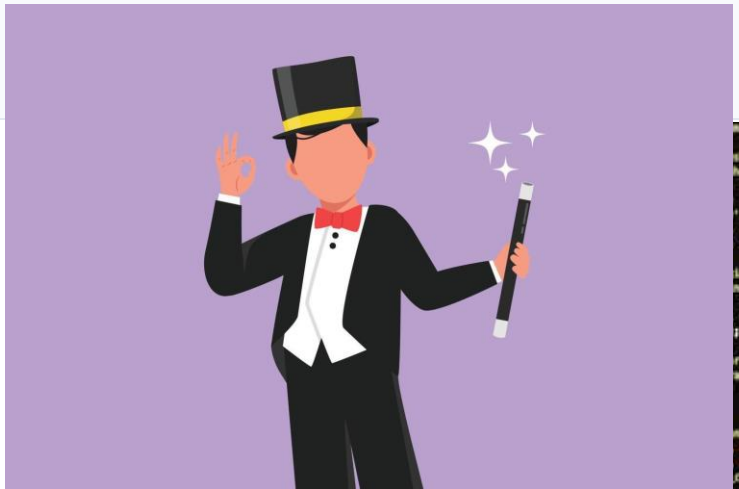
Desenhar primitivas

Prof. Dr. Raulivan Rodrigo

Contextualização

Nesta aula, vamos observar a “complexidade” da matemática pura com “simplicidade” das ferramentas de processamento gráfico.

Contudo, as bibliotecas não são "mágica", elas fornecem uma abstração sobre os algoritmos matemáticos adotados.





Desenhar primitivas

A Linha

Se tentarmos usar a equação da reta(1) em um laço for, será necessário usar números de ponto flutuante e arredondar (round()), o que custa muito processamento.

$$(1)\ y = mx + b$$

```
largura, altura = 15, 15
framebuffer = np.zeros((altura, largura))
x0, y0 = 2, 2
x1, y1 = 12, 9
passos = max(abs(x1 - x0), abs(y1 - y0))
inc_x = (x1 - x0) / passos
inc_y = (y1 - y0) / passos

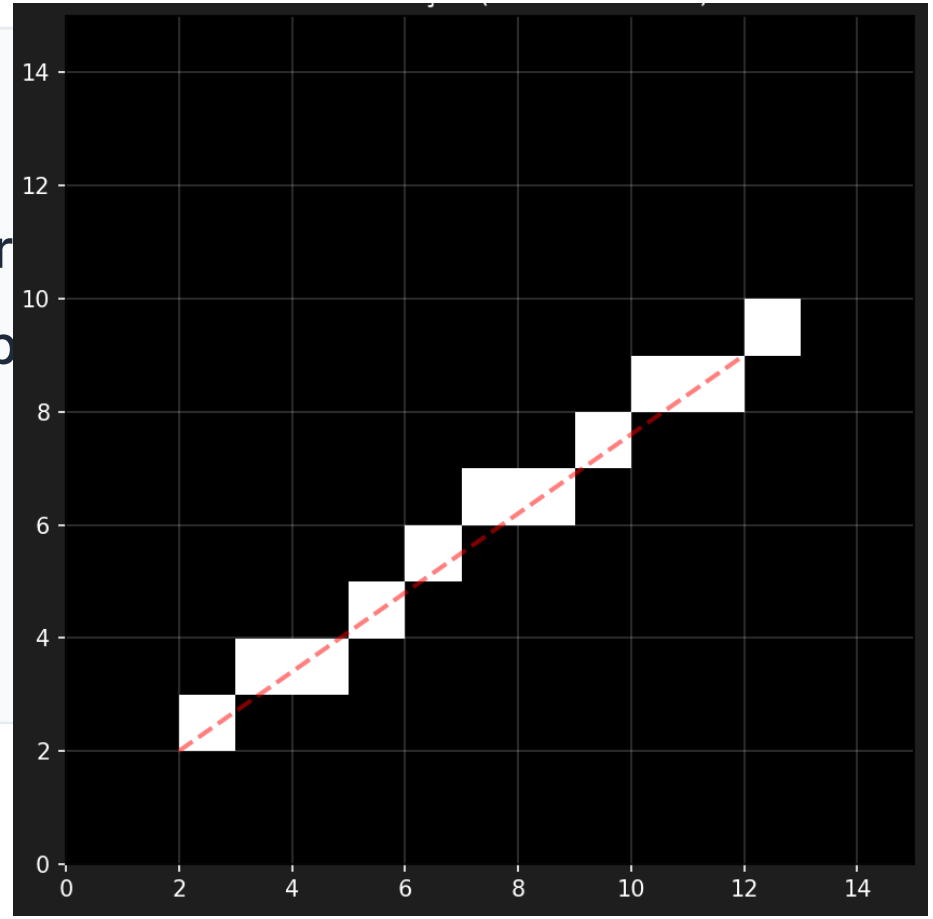
x_atual, y_atual = x0, y0

for i in range(passos + 1):
    pixel_x = int(round(x_atual))
    pixel_y = int(round(y_atual))

    framebuffer[pixel_y, pixel_x] = 1.0

    x_atual += inc_x
    y_atual += inc_y
```

Código da ultima aula rasterizacao_vetorizacao.py



Desenhar primitivas

A Linha

O Algoritmo de Bresenham (1962), desenha retas perfeitas na tela usando apenas soma e subtração de números inteiros, tornando-o absurdamente rápido no hardware.

Vamos ver o pseudocódigo

D

```
FUNCAO RasterizarLinha(x0, y0, x1, y1)
    // Calcula as distâncias absolutas (tamanho do cateto X e Y)
    deltaX <- ABS(x1 - x0)
    deltaY <- ABS(y1 - y0)

    //Define a direção do desenho (Incrementa ou Decrementa)
    SE x0 < x1 ENTAO
        passoX <- 1
    SENAO
        passoX <- -1
    FIM_SE

    SE y0 < y1 ENTAO
        passoY <- 1
    SENAO
        passoY <- -1
    FIM_SE

    //Inicializa o Acumulador de Erro
    erro <- deltaX - deltaY

    ...
```

```
...
 //Inicializa o Acumulador de Erro
 erro <- deltaX - deltaY

 //Define o ponto de partida na tela
 x <- x0
 y <- y0

 // Laço Principal de Renderização
 ENQUANTO VERDADEIRO FACA

     // Acende o pixel atual na matriz (Tela)
     PINTAR_PIXEL(x, y)

     // Condição de Parada: Se chegamos no ponto final, fim.
     SE (x = x1) E (y = y1) ENTAO
         INTERROMPER_LACO
     FIM_SE

     // O dobro do erro é usado para evitar trabalhar com divisões fracion
     erro2 <- erro * 2
     ...
```

```
...

        // O dobro do erro é usado para evitar trabalhar com divisões fracion
        erro2 <- erro * 2

        // DECISÃO DO EIXO X:
        // A reta ideal se afastou muito do centro do pixel atual na vertical
        SE erro2 > -deltaY ENTAO
            erro <- erro - deltaY
            x <- x + passoX
        FIM_SE

        // DECISÃO DO EIXO Y:
        // A reta ideal se afastou muito do centro do pixel atual na horizont
        SE erro2 < deltaX ENTAO
            erro <- erro + deltaX
            y <- y + passoY
        FIM_SE

    FIM_ENQUANTO
FIM_FUNCAO
```

Desenhar primitivas

A Linha: Observações importantes

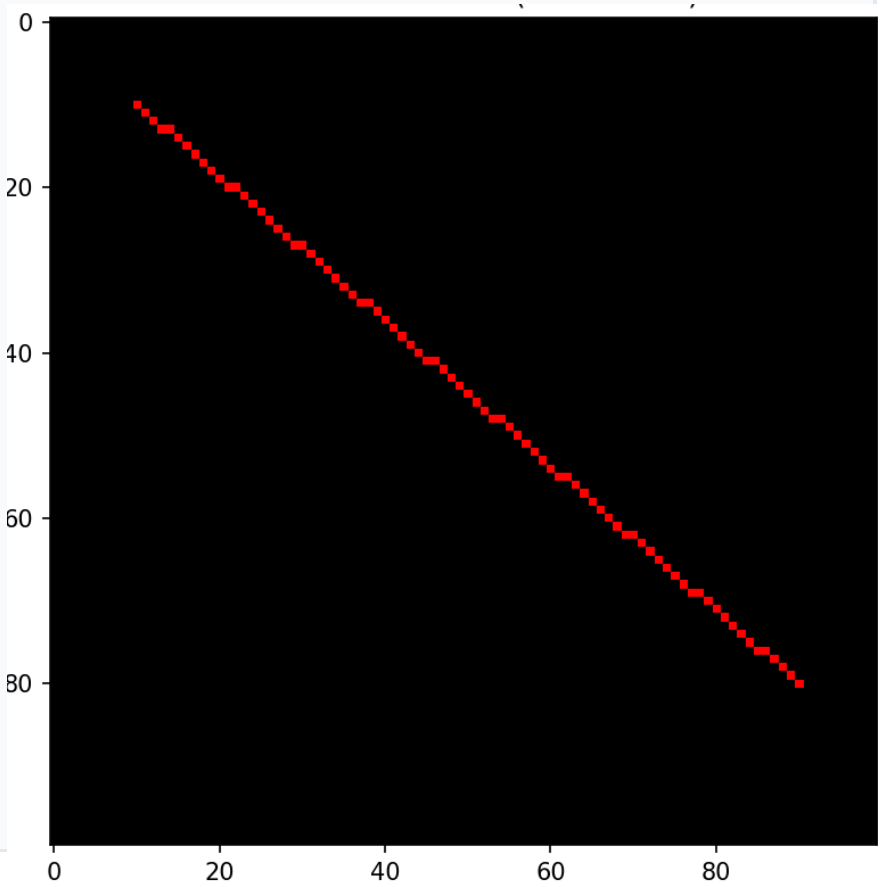
- Em vez de usar a equação da reta **y = mx + b** (onde **m** seria a divisão $\frac{\Delta y}{\Delta x}$), o algoritmo apenas mede as distâncias **deltaX** e **deltaY** como números inteiros.
- A Variável erro é o validador da reta matemática. Ela acumula a diferença entre o pixel onde estamos pisando e onde a reta ideal deveria estar.
- Já a variável erro2, a cada passo, o algoritmo testa: "Devo andar para o lado, para baixo, ou na diagonal?". Se o erro acumulado ultrapassar um limite (as condições SE), o algoritmo compensa andando no eixo adequado e "zera" parte desse erro subtraindo os deltas.

Desenhar primitivas

A Linha

Vamos implementar o algoritmo

(algoritmo_de_bresenham.py)



Desenhar primitivas

Algoritmo de Bresenham (1962)

```
1    import numpy as np
2    import matplotlib.pyplot as plt
~
```

Desenhar primitivas

```
4   def desenhar_linha_bresenham(img, x0, y0, x1, y1, cor):
5       """Algoritmo de Bresenham para rasterizar linhas."""
6       dx = abs(x1 - x0)
7       dy = abs(y1 - y0)
8
9       # direção do passo (1 ou -1)
10      sx = 1 if x0 < x1 else -1
11      sy = 1 if y0 < y1 else -1
12
13      # Variável de decisão
14      erro = dx - dy
```

Desenh

Algoritmo

```
16      x, y = x0, y0
17      # Laço infinito até chegar no ponto final
18      while True:
19          # Pinta o pixel na matriz NumPy
20          img[y, x] = cor
21
22          if x == x1 and y == y1:
23              break
24
25          e2 = 2 * erro
26          if e2 > -dy:
27              erro -= dy
28              x += sx
29          if e2 < dx:
30              erro += dx
31              y += sy
```

Desenhar primitivas

Algoritmo de Bresenham (1962)

```
33  if __name__ == "__main__":
34
35      tela = np.zeros((100, 100, 3), dtype=np.uint8) # Tela preta 100x100 RGB
36      desenhar_linha_bresenham(tela, 10, 10, 90, 80, [255, 0, 0]) # Linha vermelha
37
38      plt.imshow(tela)
39      plt.title("Reta feita 'Do Zero' (Bresenham)")
40      plt.show()
```

Desenhar primitivas

Bibliotecas Auxiliares (OpenCV)

Agora, depois de implementar toda a lógica do Bresenham.

Vamos conhecer o OpenCV pega todo esse trabalho e o encapsula em funções otimizadas em C/C++.

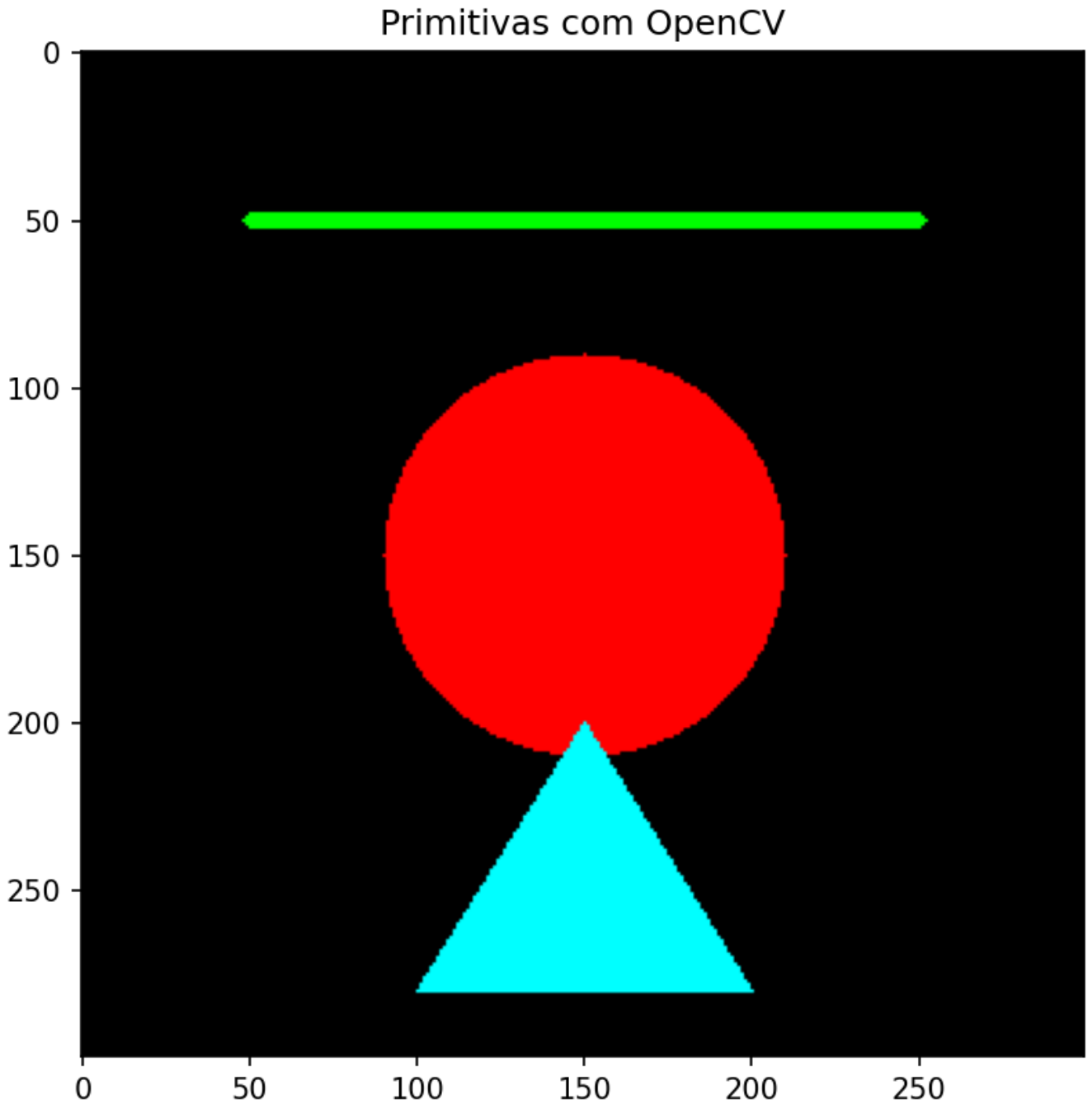
pip install opencv-python

Desenhar primitivas

Bibliotecas Auxiliares (OpenCV)

Vamos implementar o algoritmo

(algoritmo_de_bresenham_opencv.py)



Desenhar primitivas

OpenCV

```
1  import cv2
2  import numpy as np
3  import matplotlib.pyplot as plt
4
```

Desenhar primitivas

OpenCV

```
5  def main():
6      # Cria a tela (matriz NumPy)
7      tela = np.zeros((300, 300, 3), dtype=np.uint8)
8
9      # Primitiva: Linha
10     # cv2.line(imagem, ponto_inicial, ponto_final, cor_rgb, espessura)
11     cv2.line(tela, (50, 50), (250, 50), (0, 255, 0), 3) # Linha Verde
12
13     # Primitiva: Círculo
14     # cv2.circle(imagem, centro, raio, cor_rgb, espessura (-1 preenche tudo))
15     cv2.circle(tela, (150, 150), 60, (255, 0, 0), -1) # Círculo Vermelho
16
```

Desenhar primitivas

OpenCV

```
17      # Primitiva: Polígono (Triângulo)
18      pontos_triangulo = np.array([[150, 200], [100, 280], [200, 280]], np.int32)
19
20      # Só toma cuidado nessa parte pq
21      # O OpenCV precisa que o array de pontos tenha o formato (número_pontos, 1, 2)
22      pontos_triangulo = pontos_triangulo.reshape((-1, 1, 2))
23      cv2.fillPoly(tela, [pontos_triangulo], (0, 255, 255)) # Triângulo Amarelo
24
25      # Exibe a imagem
26      plt.imshow(tela)
27      plt.title("Primitivas com OpenCV")
28      plt.show()
```

Desenhar primitivas

OpenCV

```
29
30  if __name__ == "__main__":
31      main()
```

Desenhar primitivas

Existe uma divisão clássica entre o Plano Cartesiano e o Plano de Tela (Screen Space).

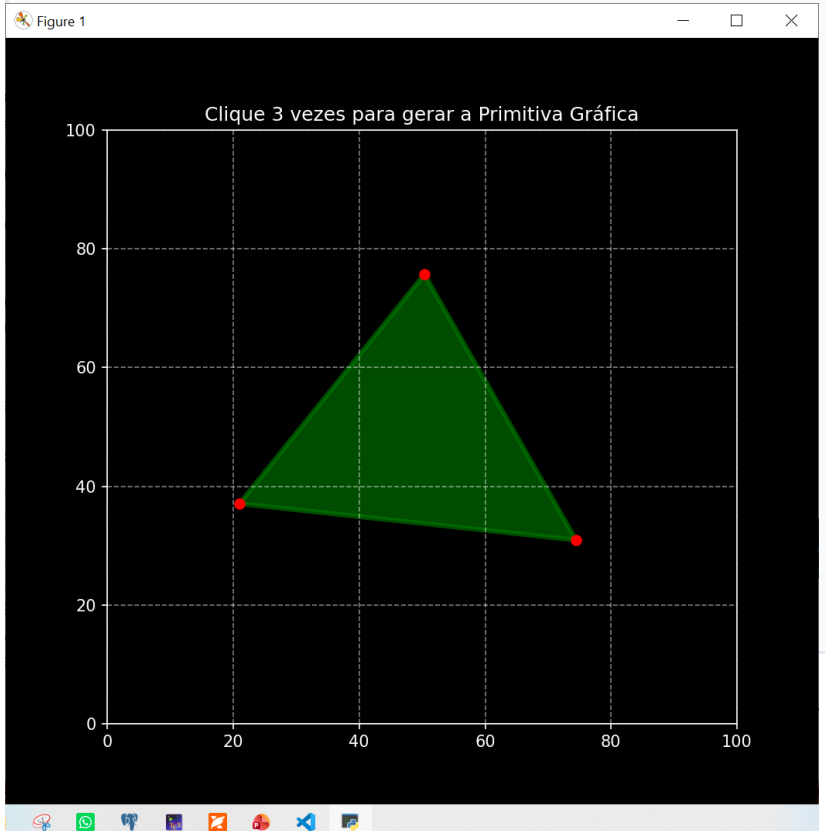
Agora que introduzimos o **OpenCV**, podemos parar de gerar as visualização das janelas pela biblioteca **matplotlib** e passaremos a usar o **cv2.imshow nativo do OpenCV** para exibir as telas.

Isso faz que o resultado apresentando seja o mesmo independente da linguagem de programação adotada, seja C, Java ou Python.

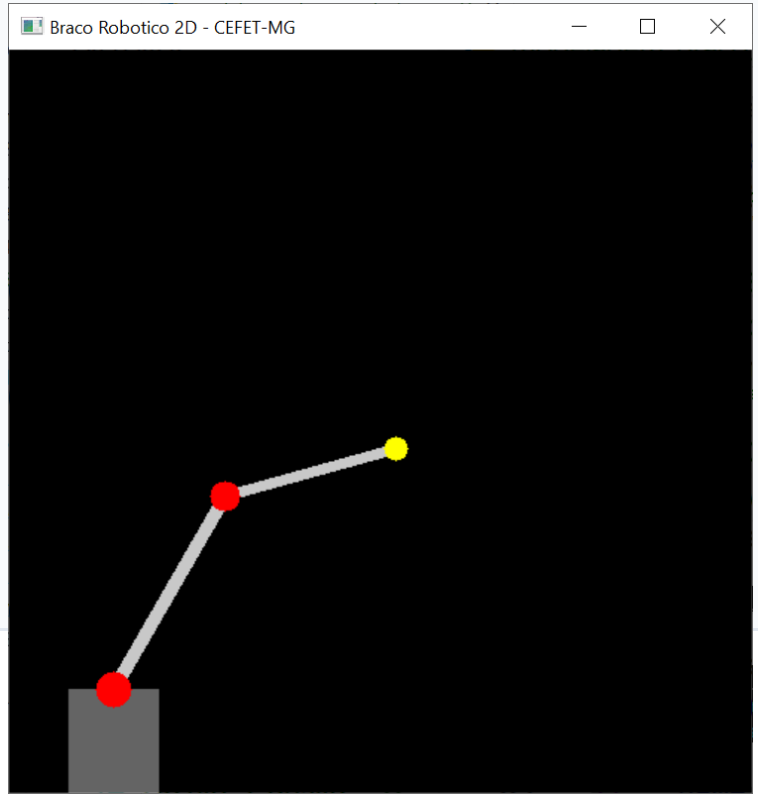
Desenhar primitivas

Existe uma divisão clássica entre o Plano Cartesiano e o Plano de Tela (Screen Space).

plt.show()



cv2.imshow("Braco Robotico 2D - CEFET-MG", img)

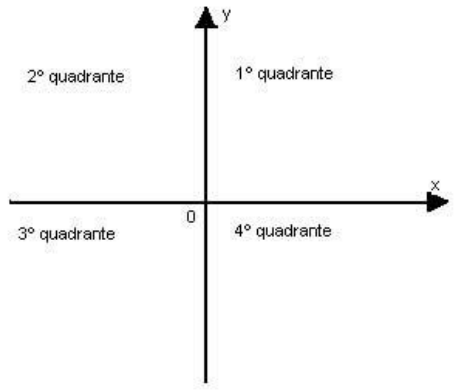


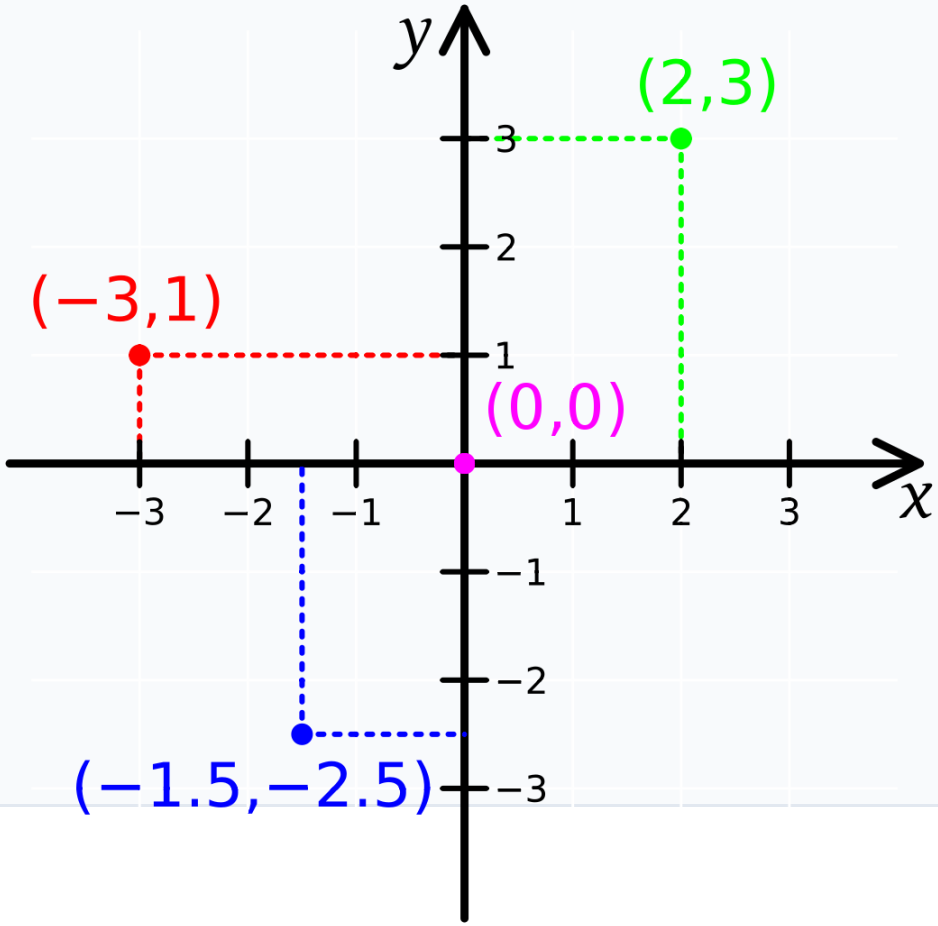
Desenhar primitivas

Existe uma divisão clássica entre o Plano Cartesiano e o Plano de Tela (Screen Space).

Monitores CRT

Aprendemos no ensino médio que no plano (René Descartes)a origem (0,0) fica no canto inferior esquerdo e o eixo Y cresce para cima.





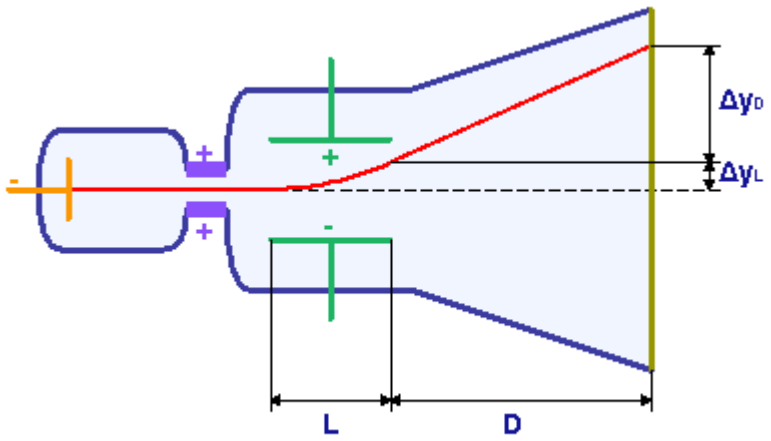
Desenhar primitivas

Existe uma divisão clássica entre o Plano Cartesiano e o Plano de Tela (Screen Space).

Monitores CRT:

Porém, quando os primeiros monitores CRT (Tubo de Raios Catódicos) foram inventados, o canhão de elétrons físicos que desenhava a imagem na tela de fósforo começava a disparar no canto superior esquerdo.





Desenhar primitivas

Existe uma divisão clássica entre o Plano Cartesiano e o Plano de Tela (Screen Space).

Monitores CRT:

Ele varria a tela da esquerda para a direita, linha por linha, de cima para baixo.

Para época, era muito mais fácil e barato que a coordenada (0,0) fosse o exato local onde o feixe de elétrons começava o seu trabalho.

Essa arquitetura virou o padrão mundial e permanece até hoje nas telas de LCD e OLED.

Desenhar primitivas

Existe uma divisão clássica entre o Plano Cartesiano e o Plano de Tela (Screen Space).

No exemplo apresentado (Algoritmo de Bresenham), usamos o matplotlib desenhar na tela e manipulamos a matriz de pixels na unha usando o **NumPy (img[y, x] = cor)**.

Uma matriz bidimensional é guardada na memória linha por linha:

- A linha 0 é a primeira linha (a que fica no topo).
- A linha 1 é a linha de baixo.
- A linha 99 é a última linha lá no fundo.

Portanto, quando o eixo Y cresce em uma matriz, você está "descendo" nas linhas.

A origem (0,0) de qualquer matriz no Python, C ou Java é sempre o canto superior esquerdo.

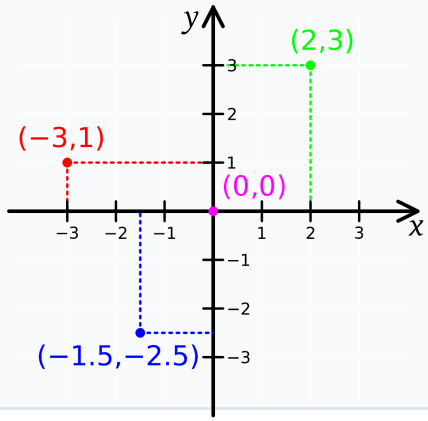
Desenhar primitivas

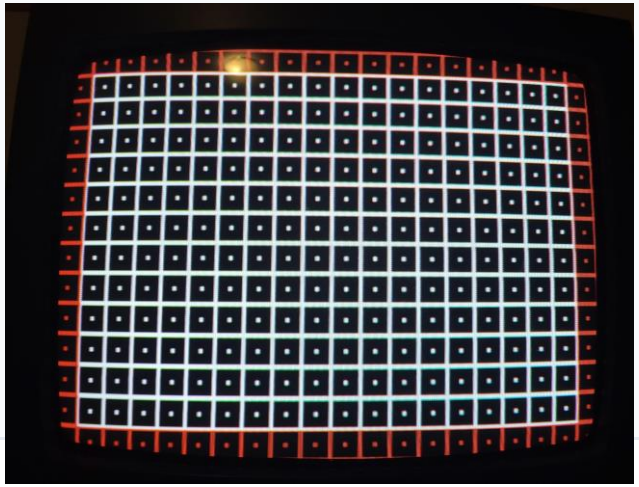
Existe uma divisão clássica entre o Plano Cartesiano e o Plano de Tela (Screen Space).

Nos exemplo apresentados nas aulas, foi usado os comandos **ax.plot()** e **patches.Polygon()**.

Esses comandos são abstrações matemáticas que o **matplotlib** cria para facilitar a nossa vida.

Ele pega o plano cartesiano clássico (Y para cima) e, nos bastidores, inverte as coordenadas antes de mandar para o monitor.





Desenhar primitivas

Existe uma divisão clássica entre o Plano Cartesiano e o Plano de Tela (Screen Space).

Já no exemplo do algoritmo de Bresenham, foi usada a função **plt.imshow(tela)**. O **imshow** significa **Image Show**.

Ele não tenta aplicar abstrações matemáticas;

Ele exibe a matriz na tela exatamente como ela está estruturada fisicamente na memória RAM (Y para baixo).

Dúvidas e Discussão

Este é o momento para aprofundarmos nos conceitos.

raulivan@cefetmg.br

SIGAA

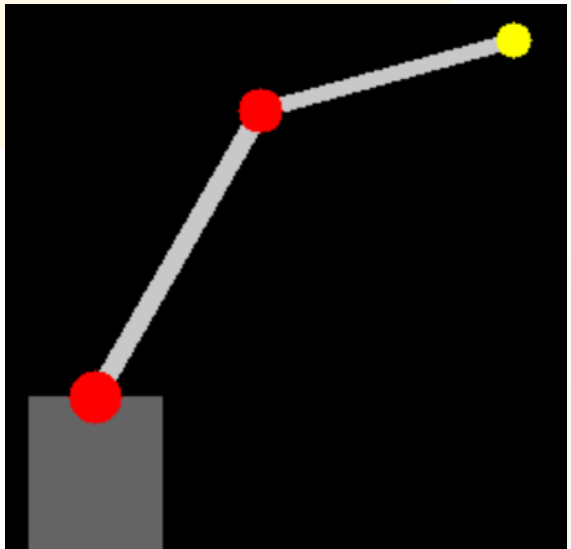
Vamos praticar

Desafio do Braço Robótico

Vamos desenhar algo que faça sentido lógico/matemático.

Calculem e desenhem um braço robótico articulado em 2D simples.

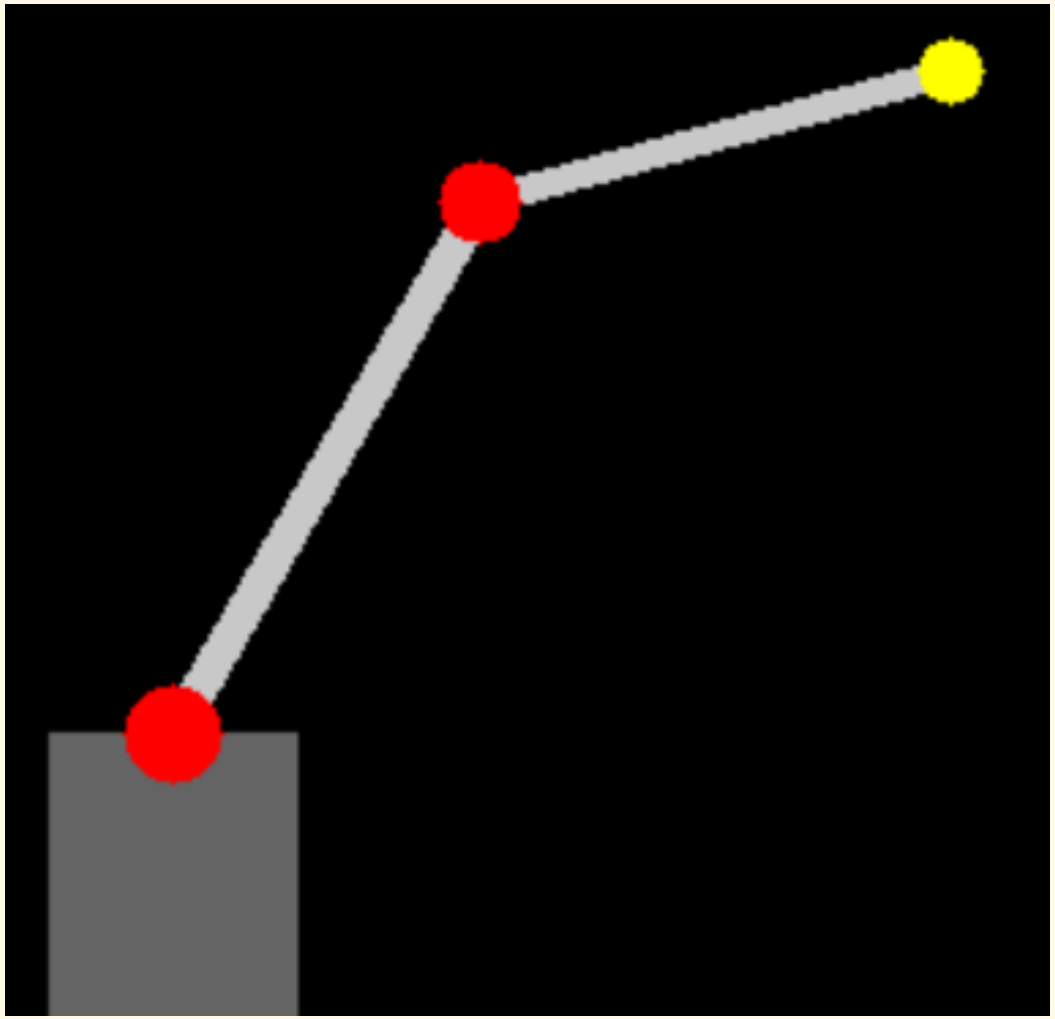
É necessário calcular as coordenadas de interseção e usar as funções do OpenCV (cv2.line, cv2.circle, cv2.rectangle) para projetar a peça na tela.



Vamos praticar

Desafio do Braço Robótico

É necessário usar trigonometria $(\cos(\theta), sin(\theta))$ para calcular onde cada junta estará no plano cartesiano a partir dos ângulos e comprimentos dos elos. E então mapear essas coordenadas matemáticas para os índices da matriz de pixels do monitor.



Vamos praticar

Desafio do Braço Robótico

Algumas dicas

A Origem ((0,0)): O **Y** cresce para baixo. Para o robô desenhar para "cima", é necessário subtrair o cálculo trigonométrico do **Y** da base. Se somar, o robô desenhará de ponta cabeça.

A soma dos ângulos: Depende do ângulo total acumulado. O ângulo do segundo elo deve ser calculado usando *math.cos(ang_ombro_rad + ang_cotovelo_rad)*. É um erro comum calcular o segundo elo usando apenas o seu próprio ângulo.

Próxima Aula

Fundamentos de Cor